

Concurrent Editing of a Record **in the UI**

Eight ways two people can want the same row — or two related rows — at the same time. Each card states the mechanism, the pros, the cons, and where it fits in a logistics platform.

THE QUESTION EVERY CARD SERVES

Two dispatchers open the same shipment. One saves. What should the other one see — a locked form, a rejected save, a merged record, or the other person's cursor?

THE RULE THAT ORDERS THE DECK

Contention is a **domain** question before it is a technical one. Pick the weakest mechanism that still protects the invariant — every step up in strength buys safety with availability, latency, or build cost.

THE EIGHT PATTERNS

- | | |
|---|---|
| ■ 01 · Pessimistic lock — hold the row while editing | ■ 05 · Real-time collaboration — CRDT / OT + presence |
| ■ 02 · Optimistic concurrency — version check on save | ■ 06 · Command / intent writes — stop editing rows |
| ■ 03 · Reservation / lease — a lock with an expiry | ■ 07 · Cascade — when one edit invalidates another |
| ■ 04 · Field-level merge — per-field last-write-wins | ■ 08 · Choosing — one screen at a time |

01

EXCLUSION

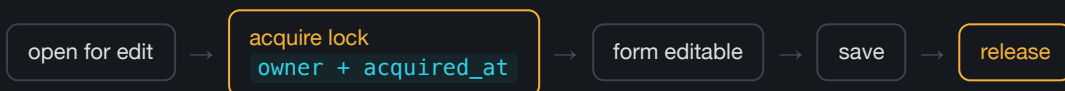
Pessimistic lock — hold the row while editing

The record is locked the moment someone opens it for edit. Everyone else gets a read-only form and a “locked by Sarah, 14:02” banner until it is released.

DECISION

Is a **conflict expensive enough** that it is better to make the second user wait than to let them work and lose it?

MECHANISM



- › The lock is **application-level and long-lived** — a row in a locks table, a KV entry, a Redis key. It is **not** a database transaction: never hold a DB transaction open across a user's think time.
- › Needs an escape hatch from day one: admin force-unlock, and release on logout / tab close.

PROS

- › **No lost work, ever.** The second user is stopped before typing, not after.
- › **Easiest to explain** to non-technical users — the mental model is a physical file on a desk.
- › No merge UI, no conflict dialog, no diffing to build.
- › Right by default where the edit is a **multi-step, high-value** transaction (rate contract, customs declaration).

CONS

- › **The lunch-break lock.** A user who wanders off leaves the record frozen — the single most-reported complaint against this pattern.
- › Needs timeouts, heartbeats and force-unlock anyway — which is Card 03, arrived at the hard way.
- › **Blocks readers-who-would-not-have-conflicted:** two users editing different fields still queue.
- › Poor fit for mobile / warehouse handhelds where sessions die without a clean release.
- › Degrades badly on hot rows — a bottleneck under load.

IN A LOGISTICS PLATFORM

- › Fits the **slow, deliberate, back-office** edits: a tariff/rate table version, a customer master record, a customs entry being prepared by one clerk.
- › Wrong for anything on the **operational hot path** — shipment status, dock slots, pick tasks — where the record is touched from many places, some of them machines.

VERDICT

Choose it only when the conflict is genuinely expensive *and* the edit is short and human. Otherwise you will end up bolting on expiry, heartbeats and force-unlock — at which point you have built Card 03 without deciding to.

strength: highest

02

DETECTION

Optimistic concurrency — check the version on save

Everyone edits freely. The version the client loaded rides along with the save, and the write is rejected if the record moved underneath it. First save wins; the second is told.

DECISION

Is contention **rare enough** that it is cheaper to detect a conflict at save time than to prevent it at open time?

MECHANISM

GET record
version 7 / ETag

→

user edits

→

PUT If-Match: 7

→

v = 7 ? write, v→8

·

else 412 / 409

- Carried as a `version` column, an `updated_at`, or an HTTP **ETag + If-Match**. `412 Precondition Failed` is the precise code for a failed conditional header; `409 Conflict` for a state conflict the request itself creates.
- The same idea, one layer down, is JetStream's `Nats-Expected-Last-Subject-Sequence` — already the write-side safety mechanism in this repo's event-sourced services.

PROS

- No blocking, no state to clean up.** Nothing is held between requests; a dead client costs nothing.
- Scales — read paths stay lock-free, so hot rows don't serialize.
- Safe across **long-lived** edits (a form open for 40 minutes) where a lock would be intolerable.
- Works uniformly for humans, APIs and integrations — an EDI partner gets the same guarantee.
- Cheap: one column, one `WHERE version = ?`.

CONS

- The loser finds out last** — after doing the work. That is the whole cost, and users feel it.
- Needs a real conflict UX. “Someone else changed this, please try again” is the failure mode people hate.
- Under high contention it thrashes: retries pile up and throughput falls below a lock.
- Whole-record versioning flags **false conflicts** — two people editing unrelated fields. (Card 04 answers this.)

MAKE THE REJECTION USEFUL

- Return **the current server state and the conflicting fields** in the error body, not just a status code — that is what lets the client auto-merge or draw a three-way diff.
- Offer the user the three real choices: **overwrite**, **discard mine**, or **merge field by field**.

VERDICT

The correct default for most business records. Its reputation is damaged almost entirely by lazy conflict UX, not by the mechanism.

strength: medium · cost: low

03

EXCLUSION, BOUNDED

Reservation / lease — a lock that expires

A soft lock with a TTL. The editor holds the record for N minutes and renews by heartbeat while the tab is alive; if the browser dies or the user leaves, the reservation lapses on its own.

DECISION

You want exclusivity, but you refuse to let a **crashed client** or an absent human freeze the record indefinitely.

MECHANISM

acquire
ttl = 120s

→ heartbeat every ~40s
renew at ⅓ TTL, with jitter

→ release on save/cancel

· no heartbeat → auto-expire

- › Renew well before expiry (a third of TTL is the common rule) and add jitter so clients don't all renew on the same tick.
- › A lease alone is **not** safe against a paused holder that wakes after expiry: pair it with a **fencing token** — a monotonically increasing number the write path validates — or with Card 02's version check as the real guard.
- › Natural fit for NATS KV: **Create** for acquire-if-absent, **Update** with revision for renew, per-bucket TTL for expiry, and **Watch** to push lock state to every other tab.

PROS

- › **Self-healing** — no stuck records, no admin force-unlock queue.
- › Keeps the pessimistic pattern's clarity ("Sarah has this until 14:07") without its worst failure.
- › Extends naturally to **business** reservations: a dock slot held for 10 minutes, capacity held while a quote is accepted.
- › Survives handheld/mobile sessions dropping — the common warehouse reality.

CONS

- › **Most moving parts of any pattern here:** TTL, heartbeat, renewal, jitter, expiry, fencing.
- › TTL is a guess. Too short interrupts real work; too long recreates the lunch-break lock.
- › Clock and pause hazards — expiry does not stop an in-flight write, so it needs a second guard anyway.
- › Adds a write-side store and a background timer to every editing screen that uses it.

IN A LOGISTICS PLATFORM

- › This is the pattern that generalises beyond editing: **reserving inventory** during checkout, **holding a dock appointment** while the carrier confirms, **claiming a pick task** so two pickers don't walk the same aisle.
- › Once a reservation is a business fact rather than a UI concern, it stops being a lock and becomes an event on the log — which is Card 06.

VERDICT

The right upgrade path from Card 01, and the right primitive for scarce physical resources. Don't build it as generic infrastructure for every screen — build it where the exclusivity is real.

strength: high · cost: high

04

RECONCILIATION

Field-level merge — send changes, not records

The client PATCHes only the fields the user actually touched. Edits to different fields merge silently; only a genuine same-field collision is a conflict.

DECISION

Is the unit of concurrency the **record**, or the **field**? Whole-record versioning declares a conflict the domain does not actually have.

MECHANISM

- › Send a **sparse patch** — dirty fields only. Untouched fields are absent, not null.
- › Default resolution is **last-write-wins per field**: the two dispatchers who changed `eta` and `driver_notes` both get their change.
- › Escalate to a conflict dialog only when the same field moved. Keep whole-record versioning as the outer guard for edits that must be **consistent across fields**.

PROS

- › **Eliminates false conflicts** — the biggest practical complaint about Card 02.
- › Most of the collaborative feel of Card 05 for a fraction of the build cost.
- › Smaller payloads, and a natural audit trail: you know which field changed and who changed it.
- › Composes with optimistic concurrency rather than replacing it.

CONS

- › **Last-write-wins silently loses data** when the same field does collide — dangerous where the value matters.
- › Breaks **cross-field invariants**: merging one user's route with another's ETA can yield a record neither person would have saved.
- › Wrong for **counters and quantities** — two clerks decrementing stock need both decrements, not the later value. That needs a counter/CRDT or a command (Card 06).
- › Requires the client to track dirty state accurately; a naive form that PATCHes everything gets none of the benefit.

THE INVARIANT TEST

- › Ask, per field: **“is this field independently meaningful?”** Free text, notes, flags, contact details — yes, merge them. Status, quantity, price, and anything two fields must agree on — no, guard the whole record.

VERDICT

Not an alternative to Card 02 — a refinement of it. Merge the independent fields, version-guard the dependent ones, and never let a quantity be resolved by last-write-wins.

strength: low-medium · cost: medium

05

CONVERGENCE

Real-time collaboration — CRDT / OT + presence

Nobody waits and nobody is rejected: edits stream between clients and converge. The Google-Docs model, applied to a business form.

DECISION

Is **simultaneous** editing a feature people are asking for — or are you buying a distributed-systems problem to avoid writing a conflict dialog?

CRDT

- › Data types whose merges commute — conflicts are **mathematically impossible** rather than resolved.
- › Offline-first falls out for free; no central transform server required, and it is the recommended starting point for new work (e.g. Yjs).
- › Cost: metadata growth, and a state model that is no longer just your row.

OPERATIONAL TRANSFORMATION

- › Concurrent operations are transformed against each other to preserve intent — the Google Docs lineage.
- › Needs a central server and correct transform functions per operation type.
- › Harder for mixed/structured content than for plain text.

PROS

- › **No conflict is ever surfaced to the user.** Best possible experience when it fits.
- › Presence, cursors and live values make contention **visible before** it becomes a conflict.
- › Offline-tolerant (CRDT) — attractive for drivers and yard staff with poor connectivity.
- › Removes the conflict-dialog design problem entirely.

CONS

- › **By far the highest complexity** — a new sync layer, new storage, new failure modes.
- › Convergence is **not correctness**: a merged form can be internally invalid. Business invariants still need a server-side gate.
- › Poor fit for records governed by a **state machine** — you cannot CRDT-merge “dispatched” and “cancelled”.
- › Auditing “who changed the ETA” gets harder — a real problem under freight/customs scrutiny.

TAKE THE CHEAP HALF

- › **Presence without CRDTs.** “Tom is viewing · Sarah is editing the ETA” costs one pub/sub subject and prevents most collisions socially. In this stack it is a `notify.*` subject and a KV watch — days of work, not quarters.
- › Adopt presence first, measure how often real conflicts remain, and only then consider merging.

VERDICT

Justified for genuinely co-authored free-form content. For a shipment record with a lifecycle, take the presence layer and leave the merge engine.

strength: n/a · cost: very high

06

AVOIDANCE

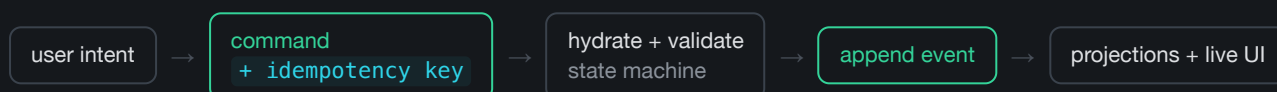
Command / intent writes — stop editing rows

The conflict exists because two users are submitting *whole records*. Submit intent instead — `reroute`, `assign driver`, `mark delivered` — and most conflicts stop being conflicts.

DECISION

Are these two users really editing the same thing, or performing **two different operations** that a CRUD form has flattened into one `PUT` ?

MECHANISM



- Two commands touching **different** aspects both succeed. Two commands touching the same one are adjudicated by **domain rules**, not by a version number — and the loser gets a meaningful message (“already assigned to Driver 12”), not “someone else saved first”.
- Each command carries an **idempotency key** so a retry or a double-tap on a handheld can never apply twice.
- Claiming a task becomes a **conditional write**: `UPDATE ... SET picker = me WHERE picker IS NULL`, or a KV `Create`. Exactly one caller wins, by construction.
- In this repo this is the existing shape: `cmd → rule → event`, guarded by `Nats-Expected-Last-Subject-Sequence` and `Nats-Msg-Id`.

PROS

- Removes most conflicts rather than resolving them** — the cheapest conflict is the one that never happens.
- Rejections are **domain-meaningful** and actionable by an operator.
- Full audit trail of who did what, in order — which freight, customs and claims work needs anyway.
- Naturally idempotent and retry-safe; correct for machine actors (EDI, telematics, scanners) too.

CONS

- Redesigns the UI** — action-shaped screens, not one big editable form. Real work, and a change users must be walked through.
- More endpoints/subjects and more domain modelling up front.
- Does not help bulk attribute editing** — someone correcting ten address fields still wants a form, and still needs Cards 02/04.
- Read models are eventually consistent; the UI must handle “accepted, not yet visible”.

VERDICT

The pattern with the highest leverage in logistics, because operational records are lifecycles, not forms. Apply it to the hot path; keep Cards 02/04 for the master-data screens behind it.

strength: strong, per-rule

07

CASCADE

When one edit invalidates another

The hardest case is not two people on one row. It is one person editing a record whose change ripples downstream — into a second record someone else already has open, and whose edit was valid when they started.

DECISION

Your write is guarded against **the record you are writing**. What guards it against the **records you read** to decide it was legal?

THE SHAPE OF THE PROBLEM

Ann opens Shipment 88
reads route R4, ETA, slot D2

→ Bob reroutes R4 →

slot D2 released,
ETA recalculated

→ Ann saves — her record
is untouched, her *premises* are not

- Card 02's version check passes — nobody else edited Shipment 88. The lost update is **invisible to a per-record guard**: the conflict is on the **read set**, not the write set. The event-sourced side calls this a **cross-aggregate invariant**.

MECHANISMS

- Read-set / dependency versioning. The form submits the versions of everything it read; the server re-checks all of them. The direct generalisation of Card 02.
- Live invalidation. The upstream change publishes a notification and every open editor holding that dependency is told *while* they work — “Route R4 changed, your ETA is stale”. Here: a `notify.*` subject plus a KV watch.
- Re-validate on save. Never trust the client's premises — re-run the domain rules against current state at write time.
- Saga / compensation. Let both land and correct forward — release the slot, re-book, notify the carrier. The honest option across service boundaries.

PROS & CONS, BY MECHANISM

- Read-set versioning — precise, no extra transport; false conflicts if the read set is drawn too wide, and someone must define “was read” per screen.
- Live invalidation — the user finds out early, when the edit is still cheap to redo; needs a dependency-to-subscriber map and does not, on its own, stop the save.
- Re-validate on save — simple, always correct to do; worst-timed rejection, and silent if the rule was never written down.
- Saga — no distributed lock, self-correcting, crosses services; compensation is a forward transition, not a rollback — it can itself fail, and intermediate states are visible.

IN A LOGISTICS PLATFORM THIS IS THE COMMON CASE, NOT THE EDGE CASE

- Change a **vessel ETA** → every booking's cut-off, every haulage slot, every customs deadline moves. Cancel an **order line** → the pick task, the load plan and the invoice are all mid-flight elsewhere. Re-rate a **tariff** → quotes already being accepted change price.
- Practical rule: **make the dependency explicit in the model**. If a record's validity depends on another record, that is a domain relationship worth naming — then invalidation, re-validation and compensation all have something concrete to hang off, instead of being rediscovered per screen.
- Decide per relationship whether the downstream edit should be **blocked**, **warned**, or **auto-corrected**. Blocking everything recreates Card 01 across the whole graph.

VERDICT

Re-validate on save as the floor; add read-set versioning where a stale premise is expensive; push live invalidation into the UI so the user learns early; reach for a saga only across services.

the case per-record guards miss

08

SELECTION

Choosing — one screen at a time

There is no platform-wide answer. Contention differs per screen, so the mechanism should too — with one house default so most screens need no decision at all.

DECISION

For this screen: how likely is a collision, and what does losing an edit actually cost?

SELECTION MATRIX

CONTENTION	COST OF A LOST EDIT	PATTERN	BECAUSE
Rare	Low — retype it	02 · Optimistic	Cheapest thing that is still correct
Rare	High — long form, legal record	02 + 04, presence on top	Merge the independent fields, warn early
Frequent	Low	06 · Commands	Different intents stop colliding
Frequent	High	03 · Lease (+ 02 as the guard)	Exclusivity that cannot get stuck
Constant, co-authored	Any	05 · CRDT/OT	Only when co-editing is the feature
Cross-record	A stale premise, not a stale row	07 · Cascade	Guard the read set, not just the write

A scarce physical resource (dock slot, stock, pick task) is not an editing question at all — it is a reservation (03) or a conditional claim (06), whatever the screen looks like.

A WORKABLE HOUSE POLICY

- › **Default:** optimistic concurrency (02) on every record, no exceptions — it is the floor, not a choice.
- › **Add** field-level patches (04) where forms are wide and fields independent.
- › **Add** presence (“who else has this open”) as a shared UI component — it prevents more conflicts than any algorithm.
- › **Escalate** to a lease (03) or a command model (06) only per screen, with a stated reason.
- › **Never** silently last-write-wins a quantity, a price, or a status.
- › **Always** re-validate against current state at write time — the cascade case (07) is invisible to a per-record version check.

FIT WITH THIS STACK

- › Optimistic writes already exist server-side as **Nats-Expected-Last-Subject-Sequence** — the UI contract is the same idea as a version/ETag.
- › Presence and lock state are a **notify.*** subject plus a KV watch — the mechanism the demo frontend already uses for live panels.
- › Leases map onto NATS KV: **Create** to acquire, revision **Update** to renew, bucket TTL to expire.
- › Commands and idempotency keys are the shape the event-sourced services are already built in.

VERDICT

Set one default, make every escalation a documented per-screen decision, and treat presence as a first-class UI concern rather than a nice-to-have.

8 cards · 1 default

Where this came from

Public sources consulted while assembling the deck, grouped by the card they informed.

01-02 · LOCKING STRATEGIES, OPTIMISTIC VS PESSIMISTIC

- › Record locking — Wikipedia · en.wikipedia.org/wiki/Record_locking
- › Record locking strategies — Progress Community · community.progress.com/s/article/P25938
- › Record and field locking strategies — HomeBase Software · hbase.net
- › Optimistic vs. pessimistic locking — Adam Djellouli · adamdjellouli.com
- › Optimistic vs pessimistic locking — Code Maze · code-maze.com

02 · VERSION CHECKS OVER HTTP

- › Handling optimistic concurrency with ETags — Ed-Fi Alliance · docs.ed-fi.org
- › Optimistic concurrency control in HTTP services — Bestmile · medium.com/bestmile
- › 409 vs 412 — Conflict vs Precondition Failed · dev-toolbox.tech

03 · LEASES, TTL AND FENCING

- › Lease pattern in distributed systems — Ajit Singh · singhajit.com
- › The lease pattern: a lock with an expiration date — Chris Woodruff · woodruff.dev
- › Leases — Kubernetes docs · kubernetes.io
- › Record-locking module spec (acquire / heartbeat / release / force-release) — open-mercato · github.com/open-mercato

04-05 · MERGE, LAST-WRITE-WINS, CRDT / OT

- › Custom conflict resolution — PowerSync docs · docs.powersync.com
- › How to implement last-write-wins — OneUptime · oneuptime.com
- › How collaborative apps handle conflicting edits — Tyler Crosse · tylercrosse.com
- › Real-time collaboration: OT vs CRDT — Tiny · tiny.cloud
- › Real differences between OT and CRDT for co-editors — arXiv · arxiv.org/pdf/1810.02137

07 · CASCADING EDITS, READ SETS AND SAGAS

- › Saga pattern for distributed transactions — Conduktor · conduktor.io
- › Mastering saga patterns — Temporal · temporal.io
- › Meeting transaction isolation with the saga pattern (lost updates, dirty reads) · medium.com
- › Saga concurrency — Particular / NServiceBus docs · docs.particular.net
- › Cooperative update exchange & read dependencies (Youtopia) — arXiv · arxiv.org/pdf/0903.5346

06 · LOGISTICS CONTEXT

- › Logistics middleware patterns for ERP / WMS / TMS integration — Sysgenpro · sysgenpro.com
- › TMS-WMS-ERP integration architecture — Locus · locus.sh
- › Pick task assignment — Oracle NetSuite WMS · docs.oracle.com
- › An uncomfortably deep dive into the idempotency key — Sameer Ahmed · medium.com